Department of Electronics & Communications

25SC2107E – Machine Learning

Tutorial Session

Logistic Regression — Binary and Multinomial Classification

Dataset: PlacementPredict | Course Outcome: CO2 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Hyderabad

1 Experiment 4: Logistic Regression — Binary and Multinomial Classification

Dataset: PlacementPredict **Course Outcome:** CO2

1.1 Aim

To build a preprocessing and modelling pipeline for the PlacementPredict dataset, to fit binary logistic regression on placement outcome and multinomial logistic regression on CGPA tier, and to evaluate both with per-class metrics rather than accuracy alone.

1.2 Learning Objectives

After completing this experiment, students should be able to:

- Combine encoding and scaling inside a single `Pipeline` that ends in a model.
- Distinguish binary from multinomial classification in the fitted output.
- Interpret the sign and size of a logistic regression coefficient.
- Read a `classification_report` and explain why accuracy alone misleads.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib scikit-learn
```

1.4 Dataset

PlacementPredict, the same `placement_predict_50k.csv` used in Tutorial 3. This experiment uses **two different answer columns**, which is the point of it.

Task	Answer column	Values	Name
1	`PlacementStatus`	Placed, NotPlaced	binary classification
2	`CGPA_Tier`	Low, Medium, High	multinomial classification

Note

Binary or multinomial? Only the number of allowed answers differs.

- **Binary** means two. The model gives one probability; the other is whatever is left over.
- **Multinomial** means three or more. One probability *per class*, always adding to 1, and the highest wins.
- You use **the same command** for both. `LogisticRegression` counts the distinct values in `y` and chooses for you.
- The evidence is the shape of `coef_`: **one row** for binary, **three rows** for three classes.

1.5 Procedure

1. Create a notebook `tut_lab4.ipynb` in this folder.
2. Load the data and inspect both answer columns.
3. Sort the input columns into numeric and text groups, and build the `ColumnTransformer`.
4. Fit binary logistic regression on `PlacementStatus` and compare against the baseline.
5. Plot the fitted coefficients and interpret them.
6. Fit multinomial logistic regression on `CGPA_Tier` and inspect `coef_`.
7. Write `evaluate_classifier()` and apply it to three models.

1.6 Notebook Implementation

Cell 1 — Import the libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

- `Pipeline` and `ColumnTransformer` are the containers from Skilling Experiment 2, now with a *model* on the end.
- Despite its name, `LogisticRegression` is a **classifier**.

Cell 2 — Load the data and look at both answers

```
placement = pd.read_csv("placement_predict_50k.csv")
print(placement.shape)

print(placement["PlacementStatus"].value_counts(normalize=True).round(3))
print()
print(placement["CGPA_Tier"].value_counts(normalize=True).round(3))
```

- About **70%** are Placed, so the dataset is **imbalanced**.
- **That 70% is the baseline.** A "model" answering "Placed" for everyone is right 70% of the time having learned nothing. For the tier task the baseline is 52%, the share of the commonest class.
- Write both baselines down *before* fitting anything.

Cell 3 — Sort the columns and build the pipeline

```
numeric = ["CGPA", "Internships", "Projects", "AptitudeScore",
           "SoftSkillsRating", "Backlogs"]
categorical = ["Stream", "Gender", "PlacementTraining"]

X = placement[numeric + categorical]
y = (placement["PlacementStatus"] == "Placed").astype(int)

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
])
```

- `StudentID` is excluded: it is a label, not a fact about the student. `CGPA_Tier` is excluded too, since it is calculated *from* `CGPA`, which is already an input.
- No `SimpleImputer` is needed, because this dataset has no missing values.
- **Counting the columns:** 6 numeric stay as 6; `Stream` (5 values) → 5; `Gender` → 2; `PlacementTraining` → 2. Total $6 + 5 + 2 + 2 = \mathbf{15}$ from the original 9.

Cell 4 — The binary model

```
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

binary_model = Pipeline([
```

```
    ("prep", preprocessor),
    ("lr", LogisticRegression(max_iter=1000)),
])
binary_model.fit(X_train, y_train)

print("Baseline (always say Placed):", round(y_val.mean(), 4))
print("Model accuracy              :",
      round(accuracy_score(y_val, binary_model.predict(X_val)), 4))
```

- `stratify=y` keeps the same 70/30 mix in both parts. Here it *is* needed, because the answer is a category.
- The `Pipeline` holds **cleaning and model together**, so one `.fit()` does all of it and you cannot scale train but forget validation.
- Baseline 0.699, model 0.807. Report it as "80.7% against a 69.9% baseline". **The gap is the achievement.**

Cell 5 — What the model learned

```
names = binary_model.named_steps["prep"].get_feature_names_out()
coefs = binary_model.named_steps["lr"].coef_[0]

pd.Series(coefs, index=names).sort_values().plot.barh(figsize=(8, 5))
plt.xlabel("coefficient")
plt.title("What raises and lowers the chance of placement")
plt.show()
```

- `coef_[0]` takes the first row. For a binary model there is only one row, which is itself worth noticing.
- **Positive** means the chance of placement rises with that input; **negative** means it falls; **size** means strength.
- Because the numbers were scaled, the bars are **directly comparable**. Without scaling they would not be.
- `CGPA` is the longest positive bar, then `PlacementTraining_Yes` and `AptitudeScore`; `Backlogs` is clearly negative. Matching common sense is evidence nothing has gone wrong.

Cell 6 — The multinomial model

```
X2 = placement[[c for c in numeric if c != "CGPA"] + categorical]
y2 = placement["CGPA_Tier"]

X2_train, X2_val, y2_train, y2_val = train_test_split(
    X2, y2, test_size=0.2, random_state=42, stratify=y2)

tier_model = Pipeline([
    ("prep", ColumnTransformer([
        ("num", StandardScaler(), [c for c in numeric if c != "CGPA"]),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
    ])),
    ("lr", LogisticRegression(max_iter=1000)),
])
tier_model.fit(X2_train, y2_train)

print("Accuracy:", round(accuracy_score(y2_val,
   tier_model.predict(X2_val)), 4))
print("Classes :", list(tier_model.named_steps["lr"].classes_))
print("coef_ shape:", tier_model.named_steps["lr"].coef_.shape)
```

- **`CGPA` must be removed here.** `CGPA_Tier` is calculated directly from it, so leaving it in

hands the model the answer. It would score near 100% having learned nothing. This is a **data leak**, the same fault as `alive` in Skilling Experiment 2.

- The command is otherwise identical to Cell 4. Accuracy is about 0.75 against a baseline of 0.52.
- `coef_.shape` prints `(3, 14)`: **3 rows**, one per class, where the binary model had one. 14 columns rather than 15 because `CGPA` was removed.

Deliverable Function

```
def evaluate_classifier(model, X_train, y_train, X_val, y_val):
    model.fit(X_train, y_train)

    train_acc = accuracy_score(y_train, model.predict(X_train))
    val_acc = accuracy_score(y_val, model.predict(X_val))

    print("Train accuracy:", round(train_acc, 4))
    print("Val accuracy  :", round(val_acc, 4))
    print()
    print(classification_report(y_val, model.predict(X_val)))

    return val_acc


models = {
    "C=0.01 (strong penalty)": LogisticRegression(C=0.01,
        max_iter=1000),
    "C=1 (default)":           LogisticRegression(C=1, max_iter=1000),
    "C=100 (weak penalty)":    LogisticRegression(C=100,
        max_iter=1000),
}

scores = {}
for name in models:
    print("=" * 55)
    print(name)
    print("=" * 55)
    pipe = Pipeline([("prep", preprocessor), ("lr", models[name])])
    scores[name] = evaluate_classifier(pipe, X_train, y_train, X_val,
        y_val)

print(pd.Series(scores).round(4))
```

One function, called three times.

- Nothing inside it is specific to logistic regression, so it works unchanged on the decision trees of Tutorial 6.
- It **prints** for the human and **returns** the validation accuracy for the next piece of code.
- `C` is the regularisation dial and works the *opposite* way to `alpha` in Skilling Experiment 4. **Small `C` means a strong penalty.**

Reading `classification_report`, taking the NotPlaced class:

- **Precision** ≈ 0.72: of all students the model *called* NotPlaced, 72% really were.
- **Recall** ≈ 0.58: of all who really were NotPlaced, it found 58%.
- **F1** ≈ 0.64 combines the two; **support** is how many were in the validation set.

Now compare the two classes. Placed scores ≈ 0.87 on F1; NotPlaced only ≈ 0.64. An overall accuracy of 0.81 hides that gap completely. The model is worse at the smaller class, which is the ordinary consequence of imbalance — **and the smaller class is usually the one you care about**. A placement cell wants to find students at risk, and 58% recall misses more

than four in ten of them.

All three `C` values land within 0.0002 of one another: with 40,000 rows and 15 features there is almost no overfitting for a penalty to correct. **Record that honestly.**

1.7 Expected Output

```
(50000, 12)
Placed       0.699        Medium     0.516
NotPlaced    0.301        Low        0.313
                          High       0.171

Baseline (always say Placed): 0.6987
Model accuracy              : 0.8065

Accuracy: 0.7537
Classes : ['High', 'Low', 'Medium']
coef_ shape: (3, 14)

C=0.01 (strong penalty)  0.8063
C=1 (default)            0.8065
C=100 (weak penalty)     0.8065
```

1.8 Result

A pipeline applying standardisation to six numeric and one-hot encoding to three nominal attributes expanded nine raw columns into fifteen features. Binary logistic regression attained a validation accuracy of 0.8065 against a baseline of 0.6987, identifying `CGPA`, placement training and aptitude score as the principal positive determinants. Multinomial logistic regression on CGPA tier, fitted after excluding `CGPA` to prevent leakage, attained 0.7537 against a baseline of 0.516 and produced a coefficient matrix of shape $(3, 14)$. Per-class analysis disclosed an F1 of 0.87 for the majority class against 0.64 for the minority, demonstrating that aggregate accuracy conceals poorer performance on the smaller and operationally more important group.

Note

The experiment in five lines

1. Write down both baselines *before* fitting: 70% for placement, 52% for tier.
2. Scale the numbers, one-hot encode the words, put the model on the end of the same pipeline.
3. Binary and multinomial use the identical command; `coef_` tells you which you got.
4. Remove `CGPA` before predicting `CGPA_Tier`, or the model is handed the answer.
5. Accuracy hides per-class failure. Always print the full report.